Group 03 - Project Report

Advanced Analytics and Applications

Team 03

Authors:
Angela Feng (7397232)
Moritz Lang (7430881)
Beate Kranz (7435471)
Lennart Jekel (7444132)
Contact: ljekel1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

List of Figures

1 Problem Description

Customer: renowned German car company Our role: top-tier management consultancy (Data Science Team of Bane & Wayne Partners (BWP))

- Our customer wants to understand the dynamics of the ride-hailing market in the US
- But the customer lacks tactical an strategic know-how in the area of operations management of electrial vehicle fleets
- Taxis are considered as a good-enough proxy for potential ride hailing vehicles

Questions to answer: - What is the goal of our analysis / report? - Data Mining Goal: - First: Describe the dynamics of the ride-hailing market in Chicago - Second: Develop a prediction model to forecast the demand - Business Goal: - Third: Thereby argue whether these dynamics and forecast make a move to the US market for our customer promising - What are the main difficulties of doing so?

2 Data Description

Analyze taxi demand patterns: Specifically show how these patterns
(start time, trip length, start and end location, price, average idle time between trips, and so on) for the given sample varies in different spatio-temporal resolution (i.e., census tract vs. varying hexagon
diameter and/or temporal bin sizes). Give possible reasons for the observed patterns.
Furthermore, can you identify spatial hot spots for trip demand using Gaussian Mixture Models (i.e.,
using Spatial Kernel Density Estimation)?

3 Data Preparation

Task: “ “ ” in other
words, your method should predict for each spatial unit (in whatever spatial layer of analysis you decide
on, e.g., hexagon or census tract) and time-basket (e.g., 08am-11.59am) the taxi demand. Important: It
is not your goal to predict hour x+1 demand given hour x demand, rather to produce an answer to questions like “what demand can we expect on a rainy Sunday from 10 – 11 am around the outskirts of
the city”. Also advise a reasonable validation strategy for your prediction model (i.e., definition of test,
training data etc). “ “ ”

- Data sources: Taxi data, weather data, Census tracts, community areas, Point of interest data
- Taxi data was downloaded from the Chicago Data Portal via API
- Full run is from first january 2024 to first may 2026
- The sample run uses one reproducible, contiguous 14-day window (`2026-04-13` through `2026-04-26`, inclusive). This preserves complete weekdays, weekends, and daily demand cycles while keeping notebook runtimes short. A 400,000-trip cap is only a safety guard; reaching it is treated as an invalid, potentially truncated sample.

- Initally a data profiling was conducted to better understand the data, find null values and plausibility issues and coverage of the Data
- This data profiling layed down the basis for the following data-prep steps
- Data was prepared following a simple bronze-silver-gold pipeline logic, whereby each dataset runs through the levels:
- Bronze represents the raw data stored as parquet file
- Silver adjusted the data quality, missing values, inadequate data, consistency checks
- Gold aggregates data, joins different data sets
- Regarding gold / generating of the temporal-spatial dataset
- First we created a dataframe containing the temporal unit, i. e. 1h, 2h or 4h. This dataframe contained nothing but the timestamps
- We integrated the weather data to this dataframe and also holidays
- Next we merged the temporal dataframe with the spatial units, i. e. census tracts, hexagons and community areas
- This gave us the keys of the final dataset and we could aggregate the taxi data into this dataframe

Thereby creating certain data charateristics that become problematic and need to be addressed during predictive tasks: - Most Temporal-Spatial units contain no or only very small demand (e. g. 3 am in some outer area of chicago) while only few contain high demand - Therefore we decided to use community area as main spatial unit

3.1 Summary of the Central Data-Preparation Steps

- Load the taxi, weather, spatial-boundary, and OpenStreetMap point-of-interest data and store the raw inputs as Parquet files in the bronze layer.
- Profile the raw data for missing values, duplicates, temporal coverage, and implausible trip, fare, and location values; use these findings to define the subsequent cleaning rules.
- Create the silver taxi dataset by removing duplicate or implausible trips, requiring valid pickup information, and retaining trips with positive fares and totals, non-negative cost components, distances of more than 0 and at most 250 miles, and durations between 1 minute and 3 hours.
- Restrict the weather observations to the taxi-data period, repair the census-tract and community-area geometries, standardize their identifiers, and generate an H3 grid covering Chicago.
- Build the gold layer by assigning taxi trips and POIs to H3 cells, census tracts, and community areas and by excluding pickup locations outside the Chicago grid.
- Construct complete 1-, 2-, and 4-hour time grids and enrich them with aggregated and imputed weather data, Illinois holiday indicators, and cyclical month, weekday, and hour features.
- Cross the temporal grids with every spatial unit, add POI counts by category, and aggregate taxi pickups as `trip_count`; explicitly retain spatial-temporal combinations with zero demand.

- Produce comparable demand datasets for all temporal and spatial resolutions, with community areas used as the principal modeling unit because finer resolutions contain a particularly high proportion of sparse or zero-demand observations.
- Create reproducible train, validation, and test datasets (approximately 70/15/15), grouping complete calendar days into the same split and checking that no dates leak across the partitions.
- 4 Datasets in silver:
  - Community area: complete data set as community area is always as spatial unit given
  - Census tracts: is more granular than community area and not always given (number of missing values), therefore we remove data without census_tract here
  - Hexagon: the hexagons are build on the census_tracts and therefore only reasonable when we have census_tract data
  - community area filtered: to make a fair comparision between the spatial units we create a filtered dataset of community area => Thereby we have one main dataset on community area and for a fair comparision of spatial units we have 3 filtered datasets

4 Data Analysis NN

- Baseline prediction
  - Zero prediction
  - Avg Spatial x Time x Weekday
  - Global median
  - see **?@tbl-trivial-baseline-test-mae**
- General validation strategy:
  - Test only for final model and comparison SVM vs
- Target and Features
  - We selected “trip_count” as target for the demand, as this will tell us how many taxi trips were started in one spatial time unit
  - Features: cyclic encoding of hour, weekday and month; weather data; holiday flag and POI information; all other taxi related columns were removed as it would otherwise lead to leakage
- Preprocessing steps:
  - Spatial Encoding is needed as the spatial units are otherwise interpreted as continiuos feature
  - StandardScaler for the feature_cols fitted on training data only

In this section we describe the approach of developing and evaluating a neural network as prediction model. Our approach is divided into three sequential steps. First we developed a simple and reproduceable baseline NN, second we performed a grid search based on the findings of the baseline NN and lastly we trained a final model based on the grid search results.

The baseline NN is trained for all spatial-units and time buckets, thereby allowing a fair comparision. Within this development phase our main goal was to determine whether NN can generally perform better compared to the baseline and which spatial-time-unit is most suited. The architecture of the baseline NN is kept simple with two hidden layers (64 and 32), ReLu activiation function and a spatial encoding for the categorical spatial units. The numerical

features were scaled using the StandardScaler, which was fitted only on the train set. In **?@tbl-nn-baseline-architecture** and **?@tbl-nn-baseline-training** we present the architecture and trainings configuration. The checkpoint-selection metric, which determines the best model troughout the epochs, is MAE on the validation set. We decided to use MAE, because it is simple and very good interpretable. However, MAE is not suitable for comparisons between different spatial-time-units. Therefore, to compare different spatial-time-units we calculate a MAE Skill Score, which is 1 - val MAE / baseline MAE, where we selected the mean spatial x time x weekday as baseline. This means that a MAE Skill Score of 0 means no improvement compared to the baseline predictions, while a positive values shows an improvement. In **?@tbl-nn-baseline-results** the results of the baseline NN are shown. The spatial-time-unit census_tract_1h is the only dataset with a positive MAE skill score, however the score is also very close to zero with 0.0043. From the table it is Further observable that the datasets on 24h time bucket are performing worst. Notebly all datasets despite community_area_24h outperform the always zero prediction. The worse performance of 24h may be due to the significant lower training size of the 24h datasets. We decided to have a closer look at the best performing model for census_tract_1h and the plots in **?@fig-nn-baseline-diagnostics** show that the model has a particular weakness for predicting higher demand.

- Show how you model's performance varies as you increase or decrease temporal or spatial resolution

?@fig-nn-baseline-all-training-histories

Therefore we decided that the main focus for the next NN should be to distinguish between zero and real demand. A first step was to switch the checkpoint-selection metric from MAE to Balanced MAE, where Balanced MAE is 0,5 * non-zero MAE + 0,5 * zero-MAE, thereby trying to ensure that zero-demand is not entirely driving the training. The second step was to perform a grid search in two stages. The first stage was exploring two strategies of dealing with the problem of zero-demand: undersampling and weighted training. The search space contains for each stragety three settings, which are shown in **?@tbl-nn-grid-stage-one**. In the first stage of the grid search the architecture was fixed to the architecture of the baseline and each model was trained for 20 epochs. We considered 20 epochs to be sufficient as it could be observed in the trainings history of the baseline model for census_tract_1h that the loss slows down decreasing at around 7 to 12 epochs (see **?@fig-nn-baseline-training-history**). The results of the first stage are shown in **?@tbl-nn-grid-performance**. Notebly all models perform worse than the spatial x time x weekday baseline, as all models have a negative MAE skill score. But this may be due to the limited trainings epochs as all runs have their best epoch very close to 20. Despite it is evidence that weighting is performing better than undersampling. All three runs with weights have higher MAE skill scores than the ones with undersampling. Therefore weighted loss is selected as strategy and used in the second stage of the grid search, where we tested different model architectures and learning rates. In **?@tbl-nn-grid-stage-two** the search space of the second stage is shown. The results of the second stage as shown in **?@tbl-nn-grid-performance** show again mainly negative MAE skill scores and best epochs close to the maximum number of epochs. The model S2-7 with the hidden layers 128-64, learning rate of 0.001 and a weight of 5 for positive demand performed best with a score of 0.0014 and balanced MAE of 2.7877. The best balanced MAE was achieved in epoch 29 of 30 (see **?@fig-nn-grid-winner-history**), which highlights that further training with more epochs may increase the performance.

The final model is therefore trained on the architecture and trainings configuration of the model S2-7 but with 100 epochs and a patience of 10. Despite giving a maximum of 100 epochs the best model was trained in epoch 26 (see **?@fig-final-census-tract-1h-training-history**) with a balanced MAE of 2.8775 and spatial x time x weekday skill score of 0.0253. The final model is therefore slightly better than the baseline and the NN baseline. The final model has a

zero-demand MAE of 0.0123 and a non-zero demand MAE of 5.7427. The large gap between these two MAE's shows that also for the final model the main difficulty is that zero-demand dominates the training and the model struggles to predict higher demand. This is also evident from the fact that the peak-demand MAE is with 17.1399 much higher than the non-zero-demand MAE and that the recall with 0.7162 is lower than the precision with 0.8123. The model struggles to predict real demand, but if it does so it is more certain. All the test-set performance metric are shown in **?@tbl-final-census-tract-1h-test-scores**.

The fact that the recall is lower than the precision also leads us to propose improvements for a follow-up project. We recommend that prior to the regressions predicts a classifier model should predict the probability of low or high demand. Further we recommend exploring separate regression models for low and high demand. The final prediction could then be an ensemble of the classification, low and high demand prediction, e. g. final prediction = p_high * high_model_pred + (1-p_high) * low_model_pred.

Despite the grid search was conducted only on the census_tract_1h dataset, we decided to train the final model S2-7 on all datasets. Of course the comparison of the final model performance between the dataset is therefore not completely fair.

```
- @fig-final-census-tract-test-mae
- @fig-final-mae-skill-trivial-baselines
```

5 SVM

To understand demand in different areas of the city, we used a neural network and SVM. To be able to compare the two models, we focused on SVM with regression. The target here, as above, is the trip demand specifically the trip count.

set non negatives to 0 since there cant be negative demand

For SVM we used the same data as described above. With time units consisting of one hour, four hour and twentyfour hour. The spatial units used include hexagons**??** The target was defined as "trip_count". The amount of started trips during The features are Features: cyclic encoding of hour, weekday and month; weather data; holiday flag and POI information; all other taxi related columns were removed as it would otherwise lead to leakage and the spatial unit encoded as latitude longitude with x, y, z due to show the dependencies of latitude and longitude. This was chosen since most other encoding did not work for all three spatial units. Especially OneHot encoding only worked for community areas. We started with a simple kernel without anything. At first we added

To compare the results with the Neural Network we decided to focus on regression instead of classification. The biggest problem we run into during the developement is the runtime of the gridsearch and the fitting. These exceeded multiple hours depending on the spatial unit and time frame.

The changes we made included: - changes we made during the developement of th svm in steps and why we decided on them - started with a basic svr on rbf kernel which is the basic kernel for scikits() implementation of svms - added grid search to find best hyperparameter - added scaler - changed grid search to only include the hyperparameter relevant for each kernel - added nystroem - changed gridsearchcv to randomsearchcv to make hyperparameter fitting quicker - changed gridsearch to include standardscaler in pipeline instead of manually - added transgressor - added memory - for linear use linear - change available hyperparamter through testing - changed to halving… instead of randomsearchcv due to better results - tested this out as the last option due to still being experimental

The last version consisted of a pipeline which is: TransformedTargetRegressor, standard-scaler, Nystroem and

- due to the data having a lot of y=0 the grid search complains about convergance issues -> this is another reason why svm for this particaluar data with h3 is not the best method
- The time frames and spatial units, used above in the Neural Network() were used here as well.
- These include time frames of one, two and twentyfour hours.
- The used spatial units include

5.1 SVR

5.2 Results

Overall the spatial units which have bigger units outside the center of the city where most people take taxis seem to perform better. This might be due to the fact that the grid searcha as well as the fitting does not

5.3 SVC

The classification was developed at the side and run on the best SVRs.

5.4 Results

5.5 How spatial resolution seems to change the outcome?

Generally

5.6 How temporal resoltion changed the outcome?

Mid temporal resoltion seems to be the best so 4h or sometimes 1 seems to overall be the best performing

5.7 Further Improvement

The first improvement we would recommend it to use data, that does not consist of a lot of trip count zero. This lead to bad convergance during the grid search as well as the model fitting. The next time we would also recommend to directly develop the model to run on GPU due to the significant amount of data. But overall we would not recommend to use SVMs for this kind of data especially for classification there are models which can handle more similar data better.

SVM

6 Spatial Encoding

7 SVR

8 SVC

9 Comparison

10 Smart Charging Reinforcement Learning

10.1 Motivation and Setup

The daily operation of electric vehicles requires them to recharge daily. Depending on the time of day, charging costs can vary significantly, especially between peak and off-peak hours. Therefore, charging cost is a significant operational expense for an electric fleet. A naive strategy of always charging at maximum power ensures the battery is full, but comes at an unnecessarily high cost since the cost grows exponentially with power. However, if we assume the vehicle is home for a fixed window, for example between 2pm and 4pm, we can be smart about when and how much to charge. With reinforcement learning we can simulate this smart charging behaviour, where an agent learns to balance cost savings against the risk of running out of energy. We model this as a Markov Decision Process (MDP), where the agent observes the current time and battery level, picks a charging power level, and receives feedback in the form of a cost signal.

Our MDP is defined as follows:

- State: the current 15-minute timestep (2:00pm, 2:15pm, ..., 4pm) and the current battery level.
- Action: one of 4 discrete charging power levels: 0 kW (off), 7 kW (low), 14 kW (medium), 22 kW (high).
- Transition: the battery increases by the chosen power $\times$ 0.25h, capped at battery capacity.
- Reward: negative charging cost each step. At the final step (4pm), a random energy demand is drawn from a normal distribution (= 30 kWh, = 5 kWh). If the battery cannot cover this demand, a large penalty is applied.

To ensure realistic simulation parameters, we grounded our assumptions in real-world data. The Tesla Model Y is one of the most commonly used models for EV taxis in North America (Kaiyi Global, 2026), and according to current EV specifications, a usable battery capacity of 60 kWh is representative of this vehicle class (EV Database, 2026). For the available charging levels, we considered Level 2 home chargers, which support power rates of up to 22 kW, sufficient for the Tesla Model Y (Pod Energy, 2026). Furthermore, we discretized the charging options into four levels: 0 kW (off), 7 kW (low), 14 kW (medium), and 22 kW (high). The starting battery at 2pm is sampled randomly between 5 and 20 kWh, reflecting a vehicle that arrives partially depleted after a morning shift. For the charging cost, we assumed home charging in Chicago, where residential electricity rates range from \$0.11 to \$0.16 per kWh (BeeCharge, 2026). We calibrated the cost coefficients to match these rates: at medium power (14 kW, delivering 3.5 kWh per 15-minute step), the coefficient ranges from 0.052 at the cheaper end to 0.076 at the more expensive end. The coefficient increases linearly across the eight timesteps, reflecting time-of-use pricing where electricity becomes more expensive during late afternoon hours.

We solve this using Q-Learning, a standard reinforcement learning method that learns optimal decisions through trial and error. Q-Learning is well suited here because the state space is small and discrete, allowing the agent to learn a complete policy without the overhead of a neural network. We additionally implemented a Deep Q-Network (DQN) to test whether a neural

network-based approach could match or improve on the tabular solution. While Q-Learning works well for our small state space, DQN would scale better to more complex scenarios, for example if the charging window were longer or if real-time electricity prices were included as an additional state variable.

10.2 Results

We trained both agents for 20,000 episodes and evaluated the learned policies over 2,000 simulated episodes against a naive baseline that always charges at maximum power (22kW). The results are summarized below:

Table 1: Evaluation results over 20000 episodes.

Policy	Avg. Cost	Savings	Avg. Battery at 4pm	Out-of-Energy Rate
Baseline (always max)	10.14	—	55.9 kWh	0.0%
Q-Learning	6.38	37.1%	47.1 kWh	0.0%
DQN	8.24	18.8%	50.2 kWh	0.0%

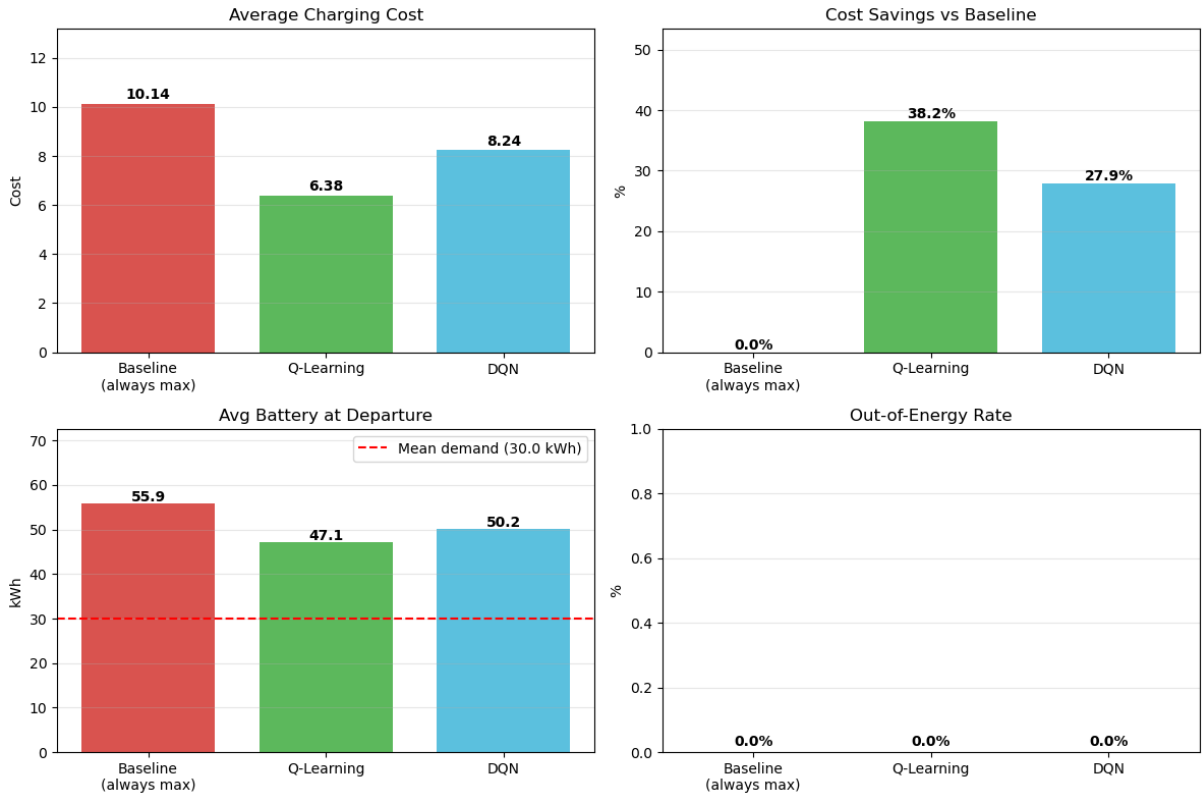

Figure 1: Comparison of charging policies: Q-Learning vs DQN vs Baseline.

The Q-Learning agent achieves a 37.1% cost reduction compared to the baseline while maintaining a 0% out-of-energy rate. It does this by charging more aggressively during the early, cheaper timesteps and reducing or skipping charging later when cost coefficients are higher. The average battery departure (47.1 kWh) comfortably exceeds the mean demand of 30 kWh, provideing a safety margin against demand uncertainty without overcharging.

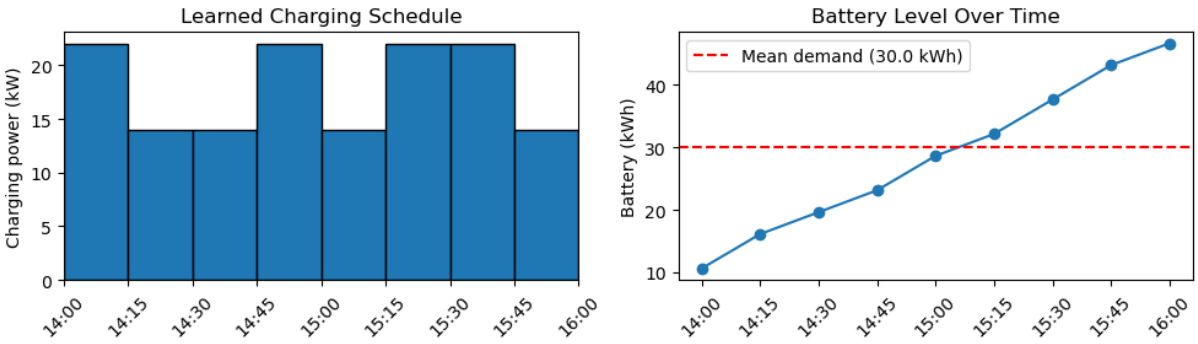

Figure 2: Learned Q-Learning charging schedule and battery trajectory.

The DQN agent also outperforms the baseline with an 18.8% cost reduction, though it charges more conservatetively than Q-Learning, resulting in a higher average battery at departures (50.2kWh). This is expected since the tabular Q-Learning approach can respresent th optimal policy exactly for our small, discrete state space. Meanwhile the DQN introduces approximation, however, its advantage lies in its scalability, as it could handle a more complex scenario with continous state variables or a longer charging window without modification.

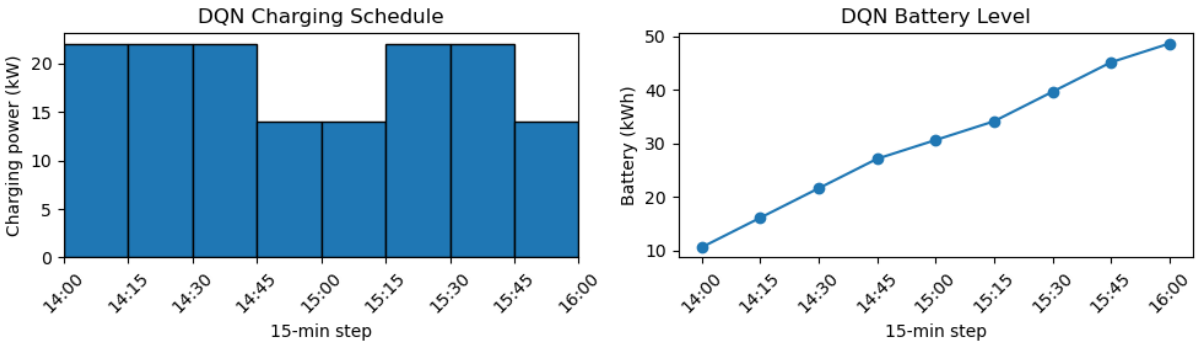

Figure 3: DQN charging schedule and battery trajectory.

All three policies achive a 0% out of energy rate, confirming that the penalty mechanism successfully teaches the agents to prioritize energy sufficiency. This is substantial, since this could leave the driver stranded and lose a full day of revenue.

The results demonstrate that even in a simplified two-hour charging window, an intelligent charging agent can reduce costs by over a third compared to a naive strategy. Scaled to an entire fleet, these savings become significant. For example, if a fleet of 100 vehicles each charges once per day, a 37% reduction in per-vehicle charging cost translates directly into lower daily operating expenses. The smart charging system is fully automated and requires no manual intervention from dirvers or dispatchers. Once trained, the agent makes real-time charging decisions based on the current battery level and time of day, adapting naturally to varying starting conditions. Moreover, the approach is flexible. If the electricity pricing structures change or the charging window shifts or the parameters are changed, the agent can simply be retrained ont he updated parameters. This makes it a future-proof component of the fleet's operational toolkit, complementing the demand prediction models developed in earlier sections of this analysis.

11 Conclusion

Template

12 Data Description

In this section, we demonstrate how to include analysis results in your report. You can either write code directly in this `.qmd` file or reference results from a separate prototyping notebook.

12.1 Prototyping in Notebooks

You can work on your analysis in the `notebooks/prototyping.ipynb` file. When you are ready to include a result, you can use Quarto's `embed` shortcode. This is a powerful feature that allows you to maintain a clean report while keeping your extensive prototyping code in separate notebooks.

To do this, you must label the specific cell in your notebook using `#| label: fig-some-name` (and optional `#| fig-cap: ...`). Then, you can embed it here like this:

Note that this requires your `.ipynb` notebooks to follow Quarto's cell-metadata syntax.

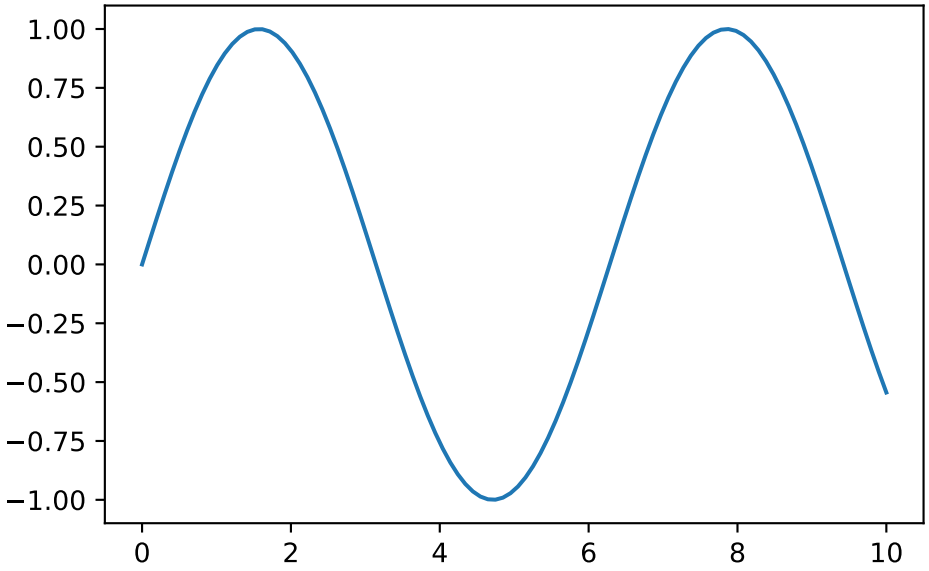

Figure 4: A sample plot generated directly in the report.

In the PDF version, the code above is hidden by default to keep the report concise and focused on the results. In the HTML version, the code can be expanded using the "Code" button.

13 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

References

BeeCharge. (2026). *Mobile ev charging cost in chicago, il*. https://www.beechargedev.com/mobile-ev-charging-cost-in-chicago-il/

EV Database. (2026). *Useable battery capacity of electric cars*. https://ev-database.org/cheatsheet/useable-battery-capacity-electric-car

Kaiyi Global. (2026). *Best ev for taxi in 2026*. https://www.kaiyiglobal.com/blog/best-ev-for-taxi-in-2026-top-5-electric-cars-for-taxi-drivers

Pod Energy. (2026). *How long to charge an electric car*. https://podenergy.com/guides/how-long-to-charge-an-electric-car

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.